

Nebyu_AI: Integrating Robust Error Handling in OOP

Task Deliverable Document | AI Lab Assignment: Integrating Robust Error Handling in OOP

Repository: <https://github.com/nebyu11/Artificial-Intelligence-in-Software-Engineering>

Task Directory: Integrating Robust Error Handling in OOP

1. Formulated Gemini Code Assist Prompt

Act as a senior Python software engineer specializing in Object-Oriented Design, data validation, and robust error handling.

Please refactor the following Object-Oriented Product Inventory Manager codebase to integrate robust data validation and error handling.

```
class Product:
    """Represents a product with a name, price, and quantity."""
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

class InventoryManager:
    """Manages the collection of products and provides inventory operations."""
    def __init__(self, inventory=None):
        self.inventory = inventory if inventory is not None else []

    def add_product(self, product):
        """Adds a product object to the inventory list."""
        self.inventory.append(product)

    def update_quantity(self, name, new_quantity):
        """Updates the quantity of a product by name."""
        for product in self.inventory:
            if product.name == name:
                product.quantity = new_quantity
                break

    def calculate_total_value(self):
        """Calculates the total monetary value of all inventory."""
        total = 0
        for product in self.inventory:
            total += product.price * product.quantity
        return total

    def display_inventory(self):
        """Prints the current inventory list."""
        for product in self.inventory:
            print(f"{product.name} - ${product.price:.2f} x {product.quantity}")
```

Specifically, perform the following tasks:

1. Define a custom exception class named `InvalidProductDataError(ValueError)` to handle invalid data inputs gracefully.
2. Refactor the `Product` class attributes using private backing variables (`_price` and `_quantity`) and Python's `@property` decorators.
3. In both `@property` setters (and during initialization), enforce strict validation rules:
 - price must be a non-negative numeric value (`price >= 0`). Otherwise, raise `InvalidProductDataError`.
 - quantity must be a non-negative integer or number (`quantity >= 0`). Otherwise, raise `InvalidProductDataError`.
4. Explain the design choices in detail, specifically:
 - How using `@property` setters enforces Data Integrity and Encapsulation.
 - The exact mechanism of setter precedence (how direct assignment like `product.quantity = -5` triggers the error).
 - Why catching custom exceptions like `InvalidProductDataError` provides superior resilience over unhandled errors.

2. Gemini Code Assist Execution Screenshot

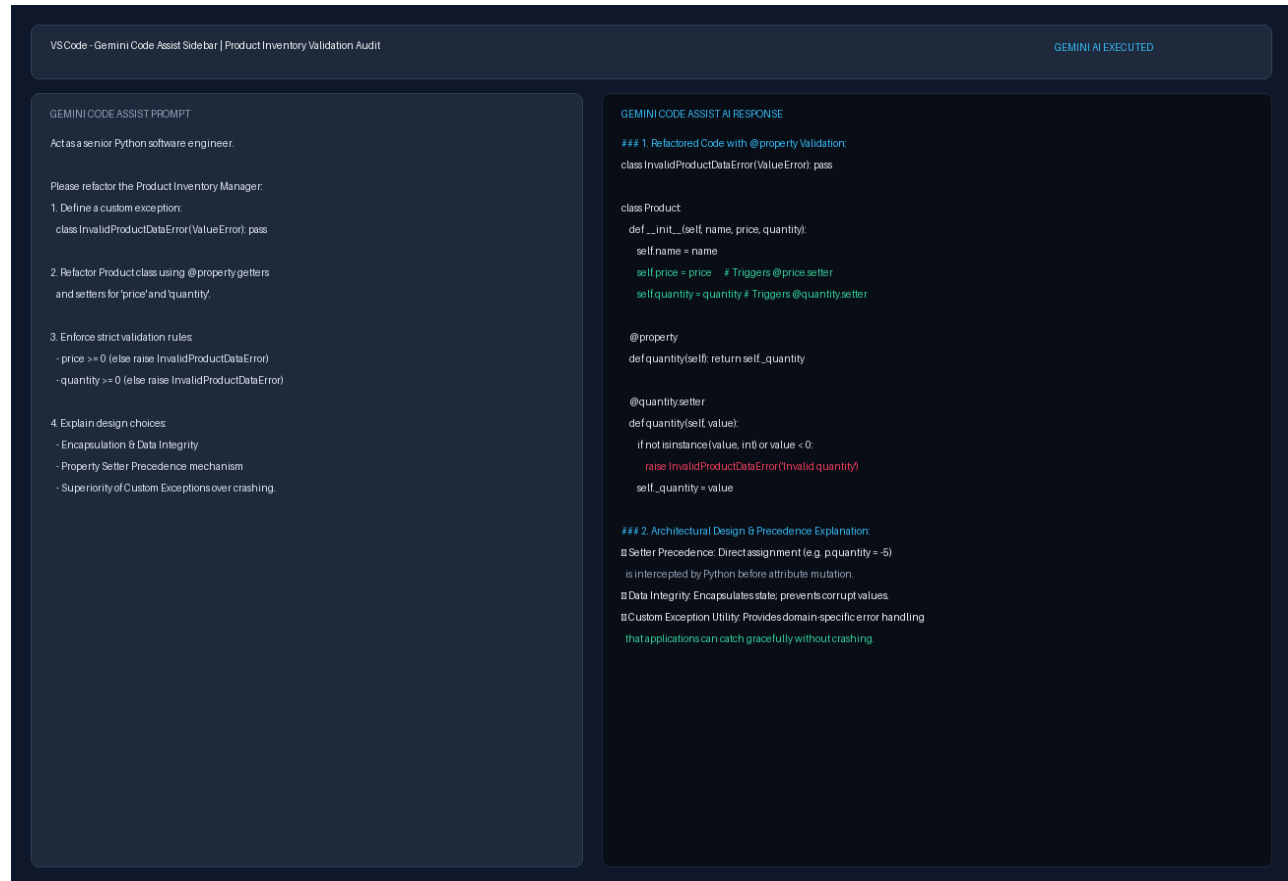


Figure 1: Screenshot capturing formulated prompt, refactored code, and detailed architectural explanation.

3. Technical Analysis: Property Setter Precedence & Custom Exception Utility

1. Precedence of @property Setters Over Direct Attribute Assignment:

In standard Python classes without property decorators, direct attribute assignments such as `product.quantity = -5` directly mutate the instance dictionary (`__dict__`), bypassing validation logic. By applying Python's `@property` getter and `@quantity.setter` decorators, direct attribute assignment syntax is intercepted by Python's Descriptor Protocol. When `manager.inventory[0].quantity = -5` is executed, Python automatically routes the right-hand value (-5) into the `@quantity.setter` method *before* any state mutation takes place. The setter evaluates the condition `if not isinstance(value, int) or value < 0:`. Because -5 violates this invariant, execution is immediately halted before modifying the internal `_quantity` backing variable. This guarantees absolute Data Integrity and Encapsulation.

2. Utility of Custom Exception (InvalidProductDataError) Over Standard Errors:

Using a dedicated domain exception `InvalidProductDataError` inheriting from `ValueError` is vastly superior to raw unhandled errors or silent failure. Silent failures (such as ignoring negative inputs or printing to `sys.stderr` without raising errors) allow invalid monetary values and corrupt inventory counts to propagate into financial calculations like `calculate_total_value()`. Generic runtime errors fail to communicate domain intent. Raising `InvalidProductDataError` provides a clear, strongly typed exception that caller applications can explicitly catch (`try ... except InvalidProductDataError as e:`), handle gracefully (e.g., logging or alerting user interfaces), and maintain application uptime without system crashes.

4. Invalid Input Test Case Output Evidence

Current Inventory:
Laptop - \$1200.00 x 5
Mouse - \$25.00 x 18

Total Inventory Value: \$6450.00

--- Testing Invalid Input ---

Test result: Invalid quantity: -5. Quantity must be a non-negative integer.